

Evaluating Historical Text Corpora

M.EP.11b and various DH/SK modules

revision of June 15, 2026

Term:	Summer 2026	Instructor:	Dr P. S. Langeslag
Time:	Mondays 10:15–11:45	Office:	SEP 2.306
Room:	VG 1.103	Office hours:	By appointment (office/online)
Credits:	See module description	E-mail:	planges@uni-goettingen.de
Prerequisites:	None for M.EP.11b	Repository:	https://github.com/langeslag/ehc

This syllabus comprises an [Overview](#) (p. 1), a [Schedule](#) (p. 1), and an annotated [Bibliography](#) (p. 10).

Overview

Course Description

In this course, students learn to use Python in the analysis of (small, premodern) text corpora using statistical and some basic machine learning methods. Each session introduces a new approach, method, or research question, which students then apply to a historical corpus of their choosing. Weekly programming assignments are submitted in the form of Jupyter notebooks and discussed in class as a way of reflecting on best practices. Students of M.EP.11b additionally each propose and carry out their own research project. No prior knowledge of Python is required.

Diversity

This course is run with the understanding that students bring a variety of backgrounds into the classroom in such domains as socioeconomics, appearance, culture, religion, ability, prior knowledge, gender, age, family situation, and sexual identity. In this course in particular, in which we will be reviewing each other's code, we need to recognize that everyone deserves equal respect from their peers regardless of their programming experience. With different backgrounds come different needs and sensitivities. If you feel your needs or those of a fellow student require special attention or are being compromised, please feel free to make this known to me by whatever channel seems most appropriate. (For more serious concerns, the University has an [independent office](#).) I will treat all requests seriously and with confidentiality, and will seek to make accommodations within my abilities and reason. At the same time, you too owe it to your fellow students to treat them with respect regardless of their background and identity. Do not stand in the way of anyone's well-being.

Schedule

Please prepare the following readings and do the following work *ahead* of the corresponding class, and take notes for in-class discussion. Upload any code to Stud.IP's Projects folder in Jupyter notebook format.

Your answers to any study questions are not to be submitted in writing; these questions serve to guide your reading focus and set the tone for class discussion. You won't be able to answer every last question, and some have no definitive answers.

Week 1 (13 April): Tools and Research Questions

Prepare:

- Set up (1) Git, (2) a Python distribution, (3) a Jupyter editor, and (4) a copy of the [course repository](#), as instructed in the course [software guide](#).

There is no other homework for our first session, but it cannot hurt to start reading up on areas that are new to you. Specifically,

- If you have no prior experience with natural language processing in Python, start on [Bird et al.](#)
- If already familiar with Python and natural language processing, start exploring premodern text corpora and experimenting on how to gain access to their XML/HTML/plain text source; you could start with those listed in the [Bibliography](#) below.

Week 2 (20 April): NLTK

Read:

- If you haven't already: [the course software guide](#)
- [The course Python guide](#)
- [Bird et al. ch. 1: "Language Processing and Python"](#) (33 pp.)

Takeaway: Walks you through querying the textbook's "Examples" corpus using the Python interpreter, and teaches the basics of Python and some widespread domains of NLP along the way.

Reading notes:

1. Some chapters (but not all) in the online edition of this textbook omit the chapter reference in headings; thus where this schedule refers to "§1.1.4," the textbook's HTML heading reads "[1.4 Counting Vocabulary](#)."
2. The exercises at the end of each chapter are optional; we won't discuss them in class. I do, however, urge you to try out all the book's code examples ("listings") given over the course of each chapter's main content as you do your weekly readings. Do **not** upload these listings as homework; for that we have our own custom assignments (marked "[Do](#):" in this schedule, starting next week).
3. The easiest way to download the textbook materials is simply to run `nltk.download('book')` instead of `nltk.download()` as described in the book; you can then skip straight to importing.
4. Please note that `nltk.FreqDist` really just reproduces the stock class `collections.Counter`, so the two may be used interchangeably. Just make sure to call it under the name by which you've imported it.

Study questions:

1. [§1.1.4](#): Keeping in mind that corpora may be in any modern or premodern language, what language features does the `lexical_diversity()` function fail to account for?
 2. [§1.5.1](#): For each of the cases of word ambiguity given under (2), can you come up with a few disambiguation rules? Use shorthand prose, not actual code, but formulate them using `if . . . then` tests that *could* straightforwardly be implemented as code.
- Students of M.EP.11b only: [project specification](#)
 - Students of M.EP.11b only: [project suggestions](#)

Further reading (optional):

- [Bird et al. ch. 2: "Accessing Text Corpora and Lexical Resources"](#) (36 pp.)
Takeaway: Introduces a range of standard online corpora, but modern, and thus beyond the scope of this course.

Week 3 (27 April): Handling Plaintext Corpora

Read:

- [Bird et al. §2.1.9 \("Loading Your Own Corpus"\)](#) (1 p., from [ch. 2: "Accessing Text Corpora and Lexical Resources"](#))
Takeaway: Introduces NLTK's `PlaintextCorpusReader`.
- [The course's demo notebook on loading and preprocessing plaintext corpora](#)
Takeaway: Explains how load a folder of text files into a corpus without the need for NLTK's `PlaintextCorpusReader`.
- [Bird et al. §§3.1–3.2, 3.4–3.5](#) (25 pp. in all, from [ch. 3: "Processing Raw Text"](#))

Takeaway: Explains how to import text data and use stock functions and regular expressions to manipulate strings.

Reading notes:

1. [§3.1 "Electronic Books"](#): This is a good exercise for learning to download remote files in Python, but in practice, it makes more sense to retrieve Project Gutenberg files using [the Project Gutenberg API wrapper](#). Also, the connection to Project Gutenberg appears rather sluggish of late; you may find the download takes 45 seconds to complete. If you encounter an `IncompleteRead` error, your connection was interrupted before the download was completed. Simply rerun your code until it works.

2. §3.1 “**Electronic Books**”: Since UTF-8 is the default encoding for read operations in Python 3, instead of `raw = response.read().decode('utf8')`, we may as well write `raw = str(response.read())`.
3. §3.1 “**Electronic Books**”: Though Project Gutenberg is thankfully accessible from Germany again, the files’ front and back matter has changed somewhat since the current revision of the book was made available. Thus to locate the start of the back matter you will want to run `raw.rfind("*** END")` rather than `raw.rfind("End of Project Gutenberg's Crime")`.
4. Under §3.1 “**Reading Local Files**,” if you are using the remote JupyterLab instance (as opposed to a local IDE) to follow along with the examples, the easiest way to create a file is to select the Text File icon in the launcher category “Other.” You can use the “right-click” context menu to rename `untitled.txt` into something more memorable after saving.
5. Also under §3.1 “**Reading Local Files**,” the `U` flag on Python’s stock function `open()`, for universal newline mode, has been deprecated and superseded by an option `newline=None`, which is set by default, as is `'r'`; so just use `f = open('document.txt')`. Note that you do have to repeat the `open()` command after running the `.read()` method, as the **garbage collector** closes it at this point. As you get more Python under your belt, you’ll learn about ways of retaining information that won’t require you to reopen files (specifically, the **with statement**).
6. §3.3 is a detailed treatment of text encoding solutions in Python. As long as we ensure we only work on UTF-8 systems and with UTF-8 files, we don’t need to worry about this, but do refer back to this section if you run into issues with non-ASCII characters.

Do:

1. Create a Jupyter Notebook experimenting with NLTK’s `PlaintextCorpusReader`, as follows:
 - (a) Duplicate `templates/fetch_repo.ipynb` into a new file in the `project/` folder, call it `03_plaintext.ipynb` but prefixed with your first name and last initial following the pattern `paul_1_03_plaintext.ipynb`, and adapt at least the first markdown cell to its new purpose. Ensure you have the `GitPython` library installed and the correct Python environment activated, then run all cells. You should now have a corpus of Old English preaching texts called **ECHOE** in the folder `corpora/echoe/`. You will find that it has `plaintext/` as well as `xml/` subfolders; for now, we’ll concern ourselves with the former.
 - (b) But to use it for NLP purposes, we’ll want to discard segment (i.e. sentence) identifiers first. Copy the code from the template `strip_identifiers.ipynb` into your new notebook (make sure to move any library imports to the first code cell) and run it there. This should yield an additional folder `corpora/echoe-bare/`. That will be our corpus for today.
 - (c) Return to Bird et al. §2.1.9 (“**Loading Your Own Corpus**”), augment your `plaintext` notebook with the necessary imports, define `corpus_root = Path.cwd().parent / 'corpora' / 'echoe-bare'`, load `.*` into `PlaintextCorpusReader` as described in the textbook instructions, and inspect the `fileids()` so you know what documents are available to call.
 - (d) Now return to Bird et al. §1.1.4 (“**Counting Vocabulary**”) and determine for a selection of ECHOE documents how their lexical diversity compares to some of the documents included with `nlk.book(text1, text2, etc.)`, running the necessary operations on the `words` property of your object, e.g. `len(wordlists.words('394.11.txt'))`; or, better, define variables like `doc1 = wordlists.words('394.11.txt')` and run your `lexical_diversity` function on them. (And instead of `wordlists`, you may want to store your corpus under its proper name, `echoe`, instead.) What strikes you about the lexical diversity of these Old English preaching texts compared to those shipped with the textbook? How do you explain your findings?
 - (e) Now head to Bird et al. §1.3.1 (“**Frequency Distributions**”), add `from nltk import FreqDist` to your imports, and plot a non-cumulative graph for the 50 most frequent word forms in three different documents. What is the most frequent term? What could be done to clean up the results? Now compare your graph with the cumulative plot from the textbook. Why do you suppose the textbook said to generate a *cumulative* graph?
 - (f) Save your `name_initial_03_plaintext.ipynb` notebook after a complete run, so all the outputs are included, and upload it to the Projects folder on Stud.IP.

Week 4 (4 May): Resource APIs and Preprocessing

Read:

- Bird et al. §§3.6–3.10 (15 pp., from ch. 3: “Processing Raw Text”)
 Takeaway: Teaches the first stages in the tackling of any text corpus with the help of regular expressions.
- The course's demo notebook on the mediawiki library.
 Takeaway: Demonstrates how to retrieve primary sources from wikis like Wikisource and Heimskringla.

Do:

1. Choose a language and matching corpus of your choosing (I will assume ECHOE as the default) and study an **overview of its inflectional morphology** alongside the rudimentary stemming syntax from the Overview slides. Now come up with a dozen or so of the most effective stemming rules for your chosen language, turn them into a stemming function based on the principles demonstrated in the slides, and apply them to a few 20-token slices of documents from your corpus.
2. Draft a markdown cell with the heading “Discussion” reflecting on the strengths and weaknesses of your stemmer in a paragraph or so.
3. Upload as 04_stemming.ipynb, prefixed with your name as in last week's assignment.

Further reading (optional):

- Bird et al. ch. 4: “Writing Structured Programs” (44 pp.)
 Takeaway: This chapter is not on NLP, but offers a more solid understanding of Python basics and is thus recommended reading for anyone with no background in Python.
- Bird et al. ch. 5: “Categorizing and Tagging Words” (34 pp.)
 Takeaway: Introduces part-of-speech tagging.

Week 5 (11 May): Parsing XML: The Document Object Model (DOM)

Watch:

- either Socratica, “XML & ElementTree” (10m), or else Max Rohowsky, “Parse XML Files with Python” (10m);
- as well as Francesco Cento, Parsing XML with Namespaces with Python (17m);
- or if this is all familiar territory to you, study the `lxml.etree` documentation instead.

Study notes:

1. All these videos describe Python's stock library for XML parsing, `xml.etree.ElementTree`. In practice, you may be better off using `lxml.etree` instead, as it has better XPath support as well as such additional functionality as the `getparent()` method. The `lxml` documentation assumes you are already familiar with `ElementTree`, however, and `lxml` namespaces behave slightly differently, see [the documentation](#).
2. The indentation issue at 8m21s and (implicitly) 8m52s of the Socratica video can be avoided by defining a **tail** attribute for the last preceding sibling.
3. If you get impatient with the namespace video, the solutions start at 13m20s.

Read:

- The course's demo notebook on the `lxml.etree` library

Do:

1. Duplicate `templates/mediawiki.ipynb` into a new file `project/05_mediawiki.ipynb` but prefixed with your first name and last initial following the pattern set out for week 3.
2. Track down a premodern or early modern document worth processing that is hosted on a Mediawiki instance, whether it be Wikisource (English, German, or otherwise; use the corresponding API), Heimskringla, or some other Mediawiki instance. Alternatively, use the Project Gutenberg API wrapper and pull content from that database.
3. Use any means necessary (BeautifulSoup if the mediawiki library won't cut it) to isolate just the part of the page you are interested in and load it into a variable.
4. Preprocess your document as needed, either into a continuous list of tokens or into verse lines, stanzas, or paragraphs each made up of a list of tokens.

5. Upload your notebook (with the output saved) into the Projects folder on Stud.IP.

Week 6 (18 May): Handling Parsed Corpora

Read:

- The course's [demo notebook on NLTK's YCOE module](#)
- The course's [demo notebook on reading parsed corpora](#)

Do:

1. Now that you know how to access XML corpora using either `xml.etree.ElementTree` or `lxml.etree`, choose an XML corpus you would like to work with and create a notebook preprocessing it into lists of tokens (you can include metadata if you like, by having each token represented by a dictionary with keys for token form and metadata). Observe the following requirements:
 - Your corpus should consist of more than one XML file.
 - If possible, embed corpus retrieval into your notebook: clone a Git repository, or use the `requests` library to download XML files directly. If this is not possible, e.g. because the corpus is distributed via the Oxford Text Archive or some other website forbidding direct downloads, write a markdown cell describing how the corpus was obtained, so readers can retrace your steps.
 - Have the container structure reflect the nature of your interest in the corpus. It is acceptable to have one list of tokens per document, but if there is any relevance to intermediate structures (sentences and verse lines above all, but perhaps paragraphs too?), have your data structure incorporate these.
 - Likewise, have the normalization standard reflect the nature of your interest in the corpus. You can keep punctuation and capitalization if you wish to present the text for reading, but any bag-of-words approach should case fold (i.e. lowercase) all tokens, as well as combining characters of similar value (e.g. `<p>` and `<ð>`).
 - Incorporate a step that writes the corpus back to disk as plaintext in a new folder below `corpora/`, one output file per document, in a presentation of your choosing.
2. Upload your notebook following the file naming format `name_initial_06_xml.ipynb`.

Further reading (optional):

- Bird et al. ch. 8: “Analyzing Sentence Structure” (30 pp.)
Takeaway: Explains how NLP may be used to parse English syntactic structures.

Week 7 (25 May): NO CLASS (English Department reading week)

Week 8 (1 June): TF-IDF

Read:

- Lane et al. ch. 3: “Math With Words (TF-IDF Vectors)” (27 pp.)
Takeaway: An introduction to the mathematical magic behind the computational analysis of relevance.

Reading notes:

1. This chapter is not very well structured, but I have yet to find a resource that combines the same depth of explanation with the same detailed code listings. You may want to read one or more of the briefer online tutorials first, so you understand the concepts going in: try [Melanie Walsh](#) or [AskPython](#), followed by [GeeksforGeeks](#).
2. I'm deliberately using the **first edition** of this textbook because the second is even harder to use. Please ensure you have the correct files.
3. Reproducing the code illustrations in this chapter is a lot of work, even if you copy and paste them from the PDF. Fortunately, the authors have made available [a notebook of of the book's code](#) you can simply download and run. However, you don't *need* to run their code; you are better off focusing on understanding the concepts the code listings. It is, at any rate, a valuable skill to be able to read code and predict what it will output!

4. If you do want to try out some of the code and are working with your own local setup, make sure to install the necessary packages (`conda install nlpia scipy scikit-learn`).
5. Just to make sure you get this right: Zipf's Law states that a word's frequency approximates one over its frequency rank ($\frac{1}{rank}$), multiplied by the number of occurrences of the most frequent word. So given that *the* ranks first, *of* second, and *and* third in frequency in a typical Modern English corpus, a document counting 100 instances of *the* should have about $\frac{1}{2} \cdot 100 = 50$ of *of*, and $\frac{1}{3} \cdot 100 = 33$ of *and*. In practice, scholars find that the relationship does not work as a power law exactly, so they add approx. 2.7 to the rank in the denominator, and also the formula works better for the "midfield" than at the head or tail of the data.
6. Errata:
 - (a) On p. 74, the authors assert that "normalized term frequency. . . [is] the word count tempered by how long the document is." This is the accepted definition of normalized TF. By their own method, however, the TF is in fact erroneously placed against the length of the list of *terms* or *types*, not the list of tokens or document length! To see the difference, compare the output of `len(bag_of_words)` with that of `len(tokens)`. In their exercise on p. 87 the authors correctly measure against the list of tokens. Then on p. 90 the mathematical expression $count(d)$ is imprecise, because this counts not the document, but rather the number of tokens it contains.
 - (b) On p. 77 for "This collections" read "This collection."
 - (c) On p. 85, for "2" in "You'll learn about part-of-speech tagging in chapter 2," read "11."
 - (d) On p. 87, ignore the variables `intro_doc` and `history_doc` given in the text body, as they don't recur.
 - (e) On p. 90, to access `log()` you'll have to import it from the `math` or `numpy` library.
 - (f) On p. 91 `query_vec` is defined twice where once would have sufficed.

• [The TF-IDF notebook from the on-corpus-tools repository](#)

Reading notes:

1. The notebook concerns *Stjórn*, a collection of biblical translations and commentaries from thirteenth- and fourteenth-century Iceland. It opens by comparing the constituent parts of the collection with each other, and then brings in a larger collection of Icelandic and Norwegian works (the Menota corpus) as a control corpus in order to investigate whether the various parts of *Stjórn* invite comparison with any particular part of the broader corpus of Old Norse–Icelandic literature.
2. Don't be confused by the notebook's unusual approach to documents. The issue is that *Stjórn* is, on the one hand, often treated as a single work despite consisting of several distinct compositions while, on the other hand, dividing naturally into biblical books; *Stjórn I* corresponds to Genesis and part of Exodus, *Stjórn II* covers the rest of Exodus and several books that follow, etc. I therefore opted to load the whole compilation as a single document and then use token indices (defined under `work_indices`) to divide it into four parts. You are unlikely to need the same approach unless your documents need separating out into multiple authors (as with the Old English prose *Genesis*, or the verse *Genesis* for that matter).
3. Among the concepts and methods the notebook incorporates are the `pandas` `DataFrame` and `matplotlib` visualization, which we will study later in the term, as well as principal component analysis (PCA), discussed in a section of Lane (§4.4) suggested for further reading next week. These are all useful tools in corpus research, so I recommend you study the notebook closely.

Do:

1. Using a parsed corpus of your choosing, load it into a list of dictionaries, one per token, so you can store part-of-speech and perhaps lemma data in addition to token form. (If your corpus of choice happens to be YCOE, you can either use NLTK's `YCOECorpusReader` or write your own code inspired on the [parsed corpora demo notebook](#).)
2. Now use what metadata you have at your disposal to come up with some way of measuring differences between your documents. For instance, you could measure the density of certain function words e.g. by taking the count of prepositions and dividing it either by the token count or by the count of nouns and personal pronouns in the document; you could measure the token length of clauses or the distance between conjunctions; or if your corpus is lemmatized, you could remove stop words and study what remaining lemmas are most frequent in each of your documents. Try to come up with one or more measures that seem meaningful to you for your corpus of choice.
3. (If you welcome the additional challenge, learn how to visualize your counts using `matplotlib`, getting you some way at least towards your assignment for week 13.)
4. Under the heading "Discussion," write a markdown cell seeking to explain what patterns you have found within your corpus, and upload your notebook to Stud.IP following the file naming format `name_initial_08_parsed_corpora.ipynb`.

5. If you can't think of a corpus or research question, consider the following. Ælfric of Eynsham produced two "series" of *Catholic Homilies*: a volume of forty texts between ca. 987 and early 991 (`cocathom1.o3` in YCOE), and a second such set (`cocathom2.o3`) between 991 and early 992. He then produced a volume of saints' lives (`coaelive.o3`) between ca. 993 and ca. 998. Scholars generally agree that Ælfric tried out a new prose style, known as rhythmical prose, towards the end of the drafting stage for his second series of homilies, which had come into its own by the time he began drafting his *Lives of Saints*. Is there some way you could quantify this development? And where do Ælfric's *Supplementary Homilies* (`coaelhom.o3`) stand on this spectrum?

Further watching (optional):

- Vsauce, "[The Zipf Mystery](#)" (21m)

Takeaway: This video makes Zipf's Law far more transparent, and offers a good deal of insight into some of its likeliest explanations.

Week 9 (15 June): Topic Analysis

Read:

- Lane et al. §§4.1–4.3.5 (26 pp., from ch. 4: "Finding Meaning in Word Counts (Semantic Analysis)")

Takeaway: A challenging introduction to how meaning may be computed.

Reading notes:

1. Just like the last time we read a chapter from Lane et al., just try to understand the concepts; don't worry about reproducing the code. Given the complexity of the material, you'll want to read this excerpt more than once.
2. Erratum: in Figure 4.3 for "TE-IDF" read "TF-IDF."

Study questions:

1. §§4.1.1–4.1.2: Having read this section as well as Lane et al. ch. 3, can you draw up a list of advantages and disadvantages of relying on TF-IDF vectors as a semantic model?
2. §§4.1.4, 4.2: Can you sum up how latent semantic analysis works in under twenty words?

- [The demo notebook on topic analysis using ECHOE](#)
- [The demo notebook on topic analysis using abstracts of scholarship](#)
- [The demo notebook on evaluating topic analysis](#)

Do:

1. Using the [TF-IDF notebook](#) from the [on-corpus-tools repository](#) for inspiration, create a TF-IDF model for a corpus of your choosing (other than Menota). Include at least the following components:
 - A cosine similarity matrix;
 - Rankings of the most similar documents for one or more selected documents of interest;
 - Rankings of the terms with the highest TF-IDF scores for one or more selected documents of interest.
2. Optionally, create 2D and 3D scatter plots using principal component analysis to convert your data to two and three dimensions, respectively.

Further reading (optional):

- Lane et al. §§4.4–4.8.2 (30 pp., from ch. 4: "Finding Meaning in Word Counts (Semantic Analysis)")

Takeaway: The remainder of the chapter offers further detail on latent semantic analysis, with emphases on principal component analysis and LDIA.

Reading note:

1. This excerpt is not supplied on Stud.IP in view of copyright restrictions.

Week 10 (22 June): Word Embeddings and Document Embeddings

Due:

- Students of M.EP.11b only: **project proposal**

Read:

- **Jurafsky and Martin ch. 5: “Embeddings”** (21 pp.)
Takeaway: An accessible, if occasionally maths-heavy, introduction to inferring meaning from the relationships between the words in a corpus.
Reading note:
 1. If you are struggling with the chapter, try reading Lynn, “**An Introduction to Word Embeddings for Text Analysis**” (3000 words) first.
- **The course’s demo notebook on word embeddings**
- **The Doc2Vec notebook from the on-corpus-tools repository**

Do:

1. Perform topic analysis (LSA/LDiA) on a corpus other than ECHOE.
2. Include a “Discussion” section in your notebook analyzing your findings.
3. Upload your notebook following the file naming format `name_initial_10_topic_analysis.ipynb`.

Further reading (optional):

- **Lynn, “An Introduction to Word Embeddings for Text Analysis”** (3000 words)
Takeaway: An accessible introduction to word vector modelling.
Reading note:
 1. Erratum? Under “Word Similarities / Synonyms,” the list of examples seems to have already been introduced in a passage that has not made it into the final text of the post.
- **Lynn, “Word Embeddings in Python With Spacy and Gensim”** (2000 words)
Takeaway: A hands-on demonstration of word vector modelling.
Reading note:
 1. The functions here based at `model.similarity()` and `model.most_similar()` have since been moved to `model.wv.similarity()` and `model.wv.most_similar()`.
- **Lane et al. ch. 6: “Reasoning With Word Vectors (Word2vec)”** (36 pp.)
Takeaway: A fuller introduction to Word2vec, GloVe, and the algebra underlying them.
Reading note:
 1. This excerpt is not supplied on Stud.IP in view of copyright restrictions.

Week 11 (29 June): *n*-Grams, Formulae, and Subword Tokenization

Read:

- **Battles, “Using N-Gram Analysis to Map Intertextual Networks in Old English Verse”** (31 pp.)
*Takeaway: A competent implementation of *n*-gram analysis.*
Reading note:
 1. If you find yourself losing focus over the probability equations, do take note of the first paragraph of p. 169: this is where Battles defines his threshold, thus turning float likelihood scores into a binary test of what counts as a poetic formula.
- **The course’s demo notebook on *n*-grams**
Takeaway: Demonstrates how to quantify the prevalence of bigrams and trigrams using off-the-shelf methods.

Do:

1. Train a Doc2Vec model on a corpus other than Menota.
2. Include a “Discussion” section in your notebook analyzing your findings.
3. Upload your notebook following the file naming format `name_initial_11_doc2vec.ipynb`.

Further reading (optional):

- [Jurafsky and Martin ch. 3: “N-gram Language Models”](#) (20 pp.)
Takeaway: Explains the probability maths behind collocations.

Week 12 (6 July): pandas DataFrames

Read:

- [The Real Python tutorial on the pandas DataFrame](#)
Takeaway: Demonstrates methods applicable to tabular data.

Do:

1. Duplicate one of your notebooks from your choice of prior assignment, but this time loop in *n*-grams at the tokenization stage (where we had previously simply used `split()`).
2. Under “Discussion,” report on the differences you notice between your original findings and the new results.
3. Upload your notebook following the file naming format `name_initial_12_n-grams.ipynb`.

Further reading (optional):

- [Lane et al, 2nd ed., Appendix C: “Vectors and Linear Algebra”](#) (16 pp.)
Takeaway: Introduces basic concepts in linear algebra with reference to NumPy arrays (equally applicable to pandas DataFrames).

Week 13 (13 July): Visualization and Statistical Principles

Read:

- [The Real Python tutorial on visualization with pandas plots](#)
Takeaway: Demonstrates how `matplotlib` plots may be generated directly from within `pandas`.
- [The Real Python guide to matplotlib](#)
Takeaway: This introduction starts from first principles and is thus less hands-on than most online tutorials, but if you stick with it, you will have a better grasp of the library for it.

Do:

1. Duplicate one of your notebooks from your choice of prior assignment, but now visualize your findings using at least two distinct kinds of plot (e.g. bar plot and scatter plot, line plot and scatter plot).
2. Upload your notebook following the file naming format `name_initial_13_visualization.ipynb`.

Further reading (optional):

- As much as you can stomach of [Oakes, “Basic Statistics”](#) (ch. 1 of *Statistics for Corpus Linguistics*; I can only provide part of the chapter)
Takeaway: Introduces basic methods in statistics, such as how to calculate standard deviation and how to establish statistical significance.

Bibliography

§1: Corpora

Clunies Ross, Margaret, Kari Ellen Gade, Guðrún Nordal, Edith Marold, Diana Whaley, and Tarrin Wills, eds. *Skaldic Poetry of the Scandinavian Middle Ages*. Accessed March 27, 2026. <https://skaldic.org>.

A current series of print editions of Old Norse poetry, as well as its digital home in the form of a collection of databases. No download available.

Faulkner, Mark, ed. *TOXIIC: Trinity Old English from the XIIth Century*. Accessed April 15, 2025. <https://mark-faulkner.com/toxiic/>.

A plaintext corpus of 12th-century English prose from three manuscripts, available by email request.

Healey, Antonette diPaolo, ed. *Dictionary of Old English Web Corpus*. Edited by John Price Wilkin and Xin Xiang. Toronto, 2024. <https://doe.artsci.utoronto.ca/?p=498>.

A 3-million-word corpus covering approximately every text (but not every manuscript witness) of Old English. A 2000 release containing HTML and SGML corpora, but not XML, is available from the Oxford Text Archive.

HeliPaD. <https://github.com/DiGS-Corpora/HeliPaD.git>.

*The C text of the Old Low German Heliand, lemmatized and marked up for parts of speech and syntax. Now also see *Helix*.*

Langeslag, P. S. *Helix: A "Heliand" Concordance*. <https://langeslag.uni-goettingen.de/helix/>.

*A further development of *HeliPaD* POS and lemmatization data to include the other manuscript witnesses, and given a web interface for querying. Click through to the *repository* for corpus data.*

McSparran, Frances, et al., eds. *Corpus of Middle English Prose and Verse*. Ann Arbor, MI. Accessed April 22, 2026. <https://quod.lib.umich.edu/c/cme/>.

An XML text corpus of a wide range of print editions; XML at <https://public.websites.umich.edu/~pfs/mec/med/>, but more conveniently at github.com/cltk/middle_english_text_cmepv.

Medieval Nordic Text Archive. Accessed March 27, 2026. <https://menota.org>.

A database of XML transcriptions of Old Norse texts. Individual XML downloads available.

Nevalainen, Terttu, et al., eds. *Corpus of Early English Correspondence*, 1998. <https://www.helsinki.fi/en/researchgroups/variation-contacts-and-change-in-english/research/corpus-of-early-english-correspondence>.

A family of corpora of English letters, 15th to 18th centuries. Available through the Oxford Text Archive.

Orchard, Andy, ed. *A Consolidated Library of Anglo-Saxon Poetry*. Accessed March 9, 2026. <https://clasp.ell.ox.ac.uk>.

An XML corpus of Old English and early Anglo-Latin poetry. Individual XML downloads available.

"Oxford Text Archive." <https://ota.bodleian.ox.ac.uk/>.

A portal providing institutional access to a range of digital corpora.

Pintzuk, Susan, Ann Taylor, Anthony Warner, Leendert Plug, and Frank Beths, eds. *The York–Helsinki Parsed Corpus of Old English Poetry*, 2001. <https://www-users.york.ac.uk/~lang18/pcorpus.html>.

A subset of DOEC, parsed for syntax and parts of speech. Available through the Oxford Text Archive.

Rissanen, Matti, et al., eds. *Helsinki Corpus of English Texts*, 2011. <https://varieng.helsinki.fi/CoRD/corpora/HelsinkiCorpus/>.

A diachronic text corpus with minimal markup except for per-text information provided in the TEI header. Available through the Oxford Text Archive.

Roelli, Philipp, ed. *Corpus corporum: Repositorium operum latinorum apud universitatem Turicensem*. Accessed March 9, 2026. <http://www.mlat.uzh.ch>.

A massive metacorpus of Latin (all eras), including Patrologia Latina and Monumenta Germaniae Historica. Individual XML downloads available.

Rudolf, Winfried, Paul Langeslag, Sabine Ines Rauch, Esther M. Lemmerz, Julia Josfeld, Irina Rau, Melanie Vollbrecht, et al., eds. "ECHOE Repository." Accessed March 27, 2026. <https://github.com/ECHOEProject/echoe>.

A 550,000-word corpus of Old English homiletic and hagiographical prose. XML and plaintext corpora directly clonable.

Samnordisk runtextdatabas. Uppsala. <https://www.nordiska.uu.se/forskn/samnord.htm>.

A database of Scandinavian runes. Download "database only" for processing.

Taylor, Ann, Anthony Warner, Susan Pintzuk, and Frank Beths, eds. *The Toronto-Helsinki-Parsed Corpus of Old English Prose*, 2003. <https://www-users.york.ac.uk/~lang22/YCOE/YcoeHome.htm>.

A subset of DOEC, parsed for syntax and parts of speech. Available through the Oxford Text Archive.

Wallenberg, Joel C., Anton Karl Ingason, Einar Freyr Sigurðsson, and Eiríkur Rögnvaldsson, eds. *Icelandic Parsed Historical Corpus*. V. 2024.03. <http://hdl.handle.net/20.500.12537/325>.

A selection of fully parsed documents, ranging from the mid-twelfth century to the present day.

§2: Theory and Programming

Bird, Steven, Ewan Klein, and Edward Loper. *Natural Language Processing with Python*. Sebastopol, CA: O'Reilly, 2009. <https://www.nltk.org/book/>.

The official book of the NLTK Python library. The HTML version is freely available CC-BY-NC-ND, and unlike the print and Kindle editions it has been updated to work with Python 3 and NLTK 3, so the code is more likely to work. However, even the online edition relies on some deprecated functions.

Deisenroth, Marc Peter, A. Aldo Faisal, and Cheng Soon Ong. *Mathematics for Machine Learning*. Cambridge: Cambridge University Press, 2020.

Eisenstein, Jacob. *Introduction to Natural Language Processing*. Cambridge, MA: MIT Press, 2019.

An excellent but challenging because purely mathematical-theoretical survey of NLP methodology.

Johnson, Kyle P. *CLTK Notebooks*. <https://github.com/cltk/cltk/tree/master/notebooks>.

Demonstrations for the current 1.0 release, not as extensive as the tutorials for the deprecated 0.1.x release.

———. *CLTK Tutorials*. <https://github.com/cltk/tutorials>.

The most extensive CLTK documentation available, though written for a deprecated 0.1.x release.

———. *The Classical Language Toolkit (CLTK): Legacy Docs*, 2019. <https://legacy.cltk.org/en/latest/>.

The somewhat sparse documentation for the deprecated 0.1.x release of CLTK.

Jurafsky, Dan, and James H. Martin. *Speech and Language Processing*. 3rd ed. draft. January 6, 2026. Accessed March 27, 2026. <http://web.stanford.edu/~jurafsky/slp3/>.

Online drafts of an outstanding textbook that does not teach programming but only the concepts and models used for NLP.

Lane, Hobson, and Maria Dyschel. *Natural Language Processing in Action*. 2nd ed. Shelter Island, NY: Manning, 2025.

The second edition of this book is not the improvement I had hoped it would be. It represents a change of focus, and a reorganization of material, and is better at some things but not necessarily at getting the concepts I want to teach across.

Lane, Hobson, Cole Howard, and Hannes Hapke. *Natural Language Processing in Action: Understanding, Analyzing, and Generating Text with Python*. 1st ed. With a foreword by Arwen Griffioen. Shelter Island, NY: Manning, 2019.

Dense and detailed, this textbook assumes a good deal of prior knowledge and isn't the best at explaining concepts. However, it is highly practical, and if you're willing to challenge yourself, it does cover a lot of ground and drives home the knowledge using practical Python exercises, as well as explaining key concepts not covered in other textbooks.

Lynn, Shane. "An Introduction to Word Embeddings for Text Analysis." <https://www.shanelynn.ie/get-busy-with-word-embeddings-introduction/>.

A short and accessible introduction.

———. "Word Embeddings in Python With Spacy and Gensim." <https://www.shanelynn.ie/word-embeddings-in-python-with-spacy-and-gensim/>.

A hands-on demonstration.

Matthes, Eric. *Python Crash Course*. 2nd ed. San Francisco, CA: No Starch, 2019.

An affordable and well-received entry-level Python textbook.

Mitchell, Melanie. *Artificial Intelligence: A Guide for Thinking Humans*. New York: Farrar, Straus and Giroux, 2019. Reprinted by Pelican 2019.

An impressively well-written introduction to the methods of and concerns over AI.

Nielsen, Michael. *Neural Networks and Deep Learning*. <http://neuralnetworksanddeeplearning.com>.

An accessible open access online (HTML+Javascript) book introducing these concepts.

Oakes, Michael P. *Statistics for Corpus Linguistics*. Edinburgh: Edinburgh University Press, 1998.

Explains the mathematical principles behind statistical significance etc.

Perkins, Jacob. *Python 3 Text Processing with NLTK 3 Cookbook*. 2nd ed. Birmingham: Packt, 2014.

Walks readers through specific tasks, such as POS tagging or custom corpus importing, using NLTK.

"Real Python." <https://realpython.com>.

An excellent collection of tutorials and video courses. You can read (most?) text-based tutorials free of charge, though you have to make a free account after a while, and at least the video tutorials rely on a pricey membership model.

Sanderson, Grant. "3Blue1Brown." <https://www.youtube.com/c/3blue1brown>.

A calculus YouTube channel with excellent explainers on the maths behind programming, including machine learning.

"The Old English Scrabble Project," September 21, 2010. <https://web.archive.org/web/20231105233605/https://wickedday.wordpress.com/2010/09/21/the-old-english-scrabble-project/>.

An account of an attempt to determine Old English character distribution in the service of Scrabble.

Vasilev, Yuli. *Natural Language Processing Using Python and spaCy: A Practical Introduction*. San Francisco: No Starch Press, 2020.

Concise but practical and valuable; focused on spaCy rather than NLTK.

"W3 Schools: Python Tutorial." <https://www.w3schools.com/python/>.

Accessible online tutorial. Not nearly as comprehensive as Real Python.

§3: Software and Resources

Bird, Steven, and Liling Tan. *NLTK: Natural Language Toolkit*. <https://www.nltk.org/>.

A Python NLP library that remains in widespread use, though it is not as advanced as spaCy.

Contributors, Project Jupyter. “JupyterLab,” 2021. <https://jupyter-cloud.gwdg.de>.

An environment for Jupyter Notebooks and Console, hosted by GWDG.

Johnson, Kyle P., Patrick J. Burns, John Stewart, Todd Cook, Clément Besnier, and William J. B. Mattingly. “The Classical Language Toolkit: An NLP Framework for Pre-Modern Languages.” In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing: System Demonstrations*, 20–29. Association for Computational Linguistics, August 2021. <https://doi.org/10.18653/v1/2021.acl-demo.3>. <https://aclanthology.org/2021.acl-demo.3>.

The writeup of the conference presentation announcing the current version of CLTK, a Python library for the processing of dead languages.

Kleinman, S., M. D. LeBlanc, M. Drout, W. Feng, and C. Zhang. *Lexos*. V. 4.0, 2019. <http://lexos.wheatoncollege.edu>.

A set of online tools for NLP analysis.

Korolev, Slava. “Neural Network to Play a Snake Game.” <https://towardsdatascience.com/today-im-going-to-talk-about-a-small-practical-example-of-using-neural-networks-training-one-to-6b2cbd6efdb3>.

A playful walkthrough of the most basic principles of machine learning.

Python Software Foundation. *Python*. V. 3.10.0, October 4, 2020. <https://www.python.org/>.

An accessible interpreted computer language often used for NLP.

Roberts, Jane, and Christian Kay, eds. *A Thesaurus of Old English*, 2017. With the help of Lynne Grundy, <https://oldenglishthesaurus.arts.gla.ac.uk>.

A thematically organized index of Old English vocabulary.

Smilkov, Daniel, and Shan Carter. “A Neural Network Playground.” <https://playground.tensorflow.org/>.

A drag-and-drop neural net simulation.

spaCy. V. 3.2. Explosion. <https://spacy.io/>.

A powerful NLP library for Python which, however, has no support for premodern languages.

Stolk, Sander. *Evoke: Exploring Vocabularies and the Concepts Their Words Evoke*. <http://evoke.uliet.net>.

A frontend for “A Thesaurus of Old English” and other data sets following the same thesaurus structure.

§4: Scholarship

“ACL Anthology.” <https://aclanthology.org/>.

An index of journal articles on NLP and computational linguistics.

Battles, Paul. “Using N-gram Analysis to Map Intertextual Networks in Old English Verse.” *Digital Philology* 8, no. 2 (2019): 155–191.

A rigorously executed study demonstrating that the poems of Cynewulf cluster closely together, and arguing that a few poems that cluster a little around it constitute a Cynewulfian sociolect.

Davis, Matthew Evan, Tamsyn Mahoney-Steel, and Ece Turnator, eds. *Meeting the Medieval in a Digital World*. Leeds: ARC Humanities Press, 2018.

A showcase of recent digital approaches to medieval literature; the contribution by Smith and Butler involves NLP.

- Gill, Paramjit S., Tim B. Swartz, and Michael Treschow. "A Stylometric Analysis of King Alfred's Literary Works." *Journal of Applied Statistics* 34, no. 10 (December 2007): 1251–1258.
- Horák, Aleš, and Adam Rambousek. "Lexicography and Natural Language Processing." In *The Routledge Handbook of Lexicography*, edited by Pedro A. Fuertes-Olivera, 179–196. London: Routledge, 2017.
- A brief overview of the value of each of these fields for the other.*
- Huang, Po-Sen, Huan Zhang, Ray Jiang, Robert Stanforth, Johannes Welbl, Jack W. Rae, Vishal Maini, Dani Yogatama, and Pushmeet Kohli. "Reducing Sentiment Bias in Language Models via Counterfactual Evaluation." *Findings of the Association for Computational Linguistics*, 2020, 65–83.
- Research into bias reduction in sentiment analysis.*
- Lewis, Joshua. "Rethinking the Value of Scrabble Tiles," December 30, 2012. <https://web.archive.org/web/20130122133051/http://blog.useost.com/2012/12/30/valett/>.
- The accompanying blog post to the release of Valett, providing a statistical grounding for Scrabble face value distribution.*
- Neidorf, Leonard, Madison S. Krieger, Michelle Yakubek, Pramit Chaudhuri, and Joseph P. Dexter. "Large-Scale Quantitative Profiling of the Old English Verse Tradition." *Nature Human Behavior* 3 (6 2019): 560–567.
- Norvig, Peter. "English Letter Frequency Counts: Mayzner Revisited; or ETAOIN SRHLDU." Accessed March 27, 2026. <http://norvig.com/mayzner.html>.
- An accessible read on letter distribution in Present-Day English.*
- Plecháč, Petr, Andrew Cooper, Benjamin Nagy, and Artjoms Šeļa. "Beowulf Single-Authorship Claim Is Unsupported." *Nature Human Behavior* 5 (2021): 1481–1483.
- A critical response to Neidorf et al.*
- Smith, William H., and Charles L. Butler. "Statistical Analysis and the Boundaries of the Genre of Old English Prayer." In Davis, Mahoney-Steel, and Turnator, *Meeting the Medieval in a Digital World*, 11–26.
- Applies relative term frequency to compare Old English prayers.*
- Stolk, Sander, and Thijs Porck, eds. "Exploring Early Medieval English Eloquence: A Digital Humanities Approach with 'A Thesaurus of Old English' and Evoke." *Amsterdamer Beiträge zur älteren Germanistik* 81.
- A special issue on the Evoke application and its Old English data sets.*
- Torabi, Katayoun. "If (Not 'Quantize, Click, and Conclude'): {Digital Methods in Medieval Studies}." In Davis, Mahoney-Steel, and Turnator, *Meeting the Medieval in a Digital World*, 27–44.
- A survey arguing that available tools do not replace close reading.*
- Wunderlich, Martin, Alexander Fraser, and P. S. Langeslag. "'God wat þæt ic eom god': An Exploratory Investigation into Word Sense Disambiguation in Old English." In *Proceedings of GSCL 2015: International Conference of the German Society for Computational Linguistics and Language Technology*, 39–48. Munich: Gesellschaft für Sprachtechnologie und Computerlinguistik, 2015. <https://konvens.org/proceedings/2015/>.
- Tests a selection of supervised approaches for sense disambiguation for Old English.*
- Younes, Nadja, and Ulf-Dietrich Reips. "Guideline for Improving the Reliability of Google Ngram Studies: Evidence from Religious Terms." *PLoS ONE* 14 (3 2019). <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0213554>.
- Zhang, Sarah. "The Pitfalls of Using Google Ngram to Study Language." *Wired*, October 12, 2015. <https://www.wired.com/2015/10/pitfalls-of-studying-language-with-google-ngram/>.